

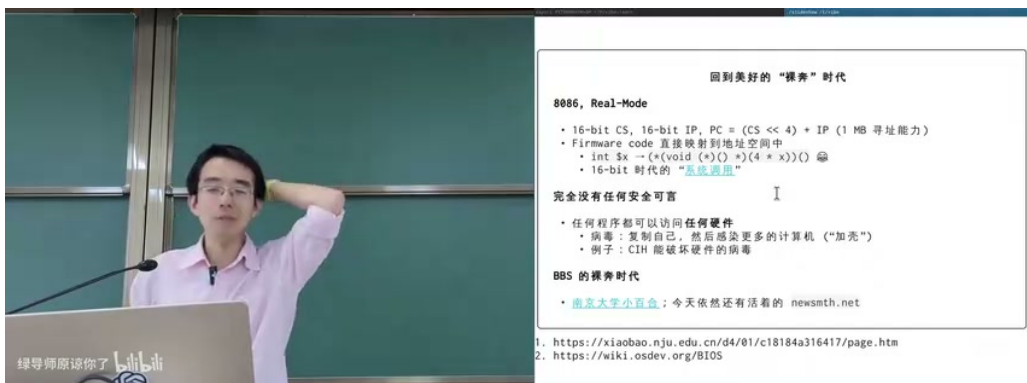
课程笔记

28 - 计算机 systems 安全

2026 南京大学操作系统原理

lecture-to-notes

2026-06-20



回到美好的“裸奔”时代

8086, Real-Mode

- 16-bit CS, 16-bit IP, $PC = (CS \ll 4) + IP$ (1 MB 寻址能力)
- Firmware code 直接映射到地址空间中
 - `int $x → (*(void (*)()) (*(4 * x)))()` ☹
 - 16-bit 时代的“系统调用”

完全没有任何安全可言

- 任何程序都可以访问任何硬件
 - 病毒：复制自己，然后感染更多的计算机（“加壳”）
 - 例子：CIH 能破坏硬件的病毒

BBS 的裸奔时代

- [南京大学小百合](https://xiaobao.nju.edu.cn/d4/01/c18184a316417/page.htm)；今天依然还有活着的 news.mth.net

1. <https://xiaobao.nju.edu.cn/d4/01/c18184a316417/page.htm>
2. <https://wiki.osdev.org/BIOS>

视频作者/频道：南京大学操作系统原理：蒋炎岩

发布日期：2026

视频时长：01:24:13

视频链接：本地视频文件

目录

1	为什么操作系统课要谈安全	3
1.1	从“裸奔”的早期系统说起	3
1.2	CIA：安全目标的三个维度	3
1.3	本章小结	4
2	密码学只讲够用的一分钟	5
2.1	单向函数与摘要	5
2.2	陷门单向函数：加密、认证与签名	5
2.3	本章小结	6
3	操作系统的安全抓手：访问控制	6
3.1	系统调用既是功能边界，也是权限边界	6
3.2	访问控制矩阵	7
3.3	UID、GID、root 与 setuid	8
3.4	/etc/passwd 与路径竞争	9
3.5	从矩阵切片：ACL、AppArmor、seccomp 与 BPF	9
3.6	本章小结	10
4	程序漏洞：访问控制挡不住“意外路径”	11
4.1	Fuzz testing：自动寻找非预期行为	11
4.2	Stack smashing：一个经典提权路径	11
4.3	防御：缩小攻击面与维护控制流完整性	13
4.4	本章小结	13
5	侧信道：没有越权访问，也可能泄密	14
5.1	Timing side channel：密码逐字节泄漏	14
5.2	Meltdown：投机执行留下缓存痕迹	15
5.3	物理侧信道：屏幕、电磁与冷启动	15
5.4	本章小结	16
6	Agent 时代的新安全问题	16
6.1	Prompt injection：自然语言里的控制流劫持	16
6.2	供应链攻击：你甚至不能信任你的软件	17
6.3	本章小结	18
7	总结与延伸	18
7.1	四层安全图景	18
7.2	给操作系统学习者的具体 takeaways	19
7.3	拓展阅读	19

7.4 最终小结	19
--------------------	----

1 为什么操作系统课要谈安全

这一讲不是传统操作系统课程主线中的考核内容，而是把前面学过的进程、地址空间、文件系统、系统调用和资源管理重新放到“恶意使用”的坐标系中。正常的 OS 课程常问“这个 API 能做什么”，安全问题追问的是：当调用者不可信、输入不可信、网络不可信、依赖不可信，甚至硬件实现也不完全可信时，系统还能保证什么？

1.1 从“裸奔”的早期系统说起

早期 8086 real mode PC 没有今天意义上的内存保护。16-bit 的 CS 和 IP 组合出 20-bit 物理地址，程序可以直接访问 1MB 地址空间中的任意位置；系统调用也只是通过 `int` 指令查中断向量表，跳转到固件中的处理例程。换句话说，CPU 和系统并没有在程序之间建立强制隔离：一个程序可以修改另一个程序的内存、补丁代码、感染可执行文件，甚至破坏硬件。

安全意识的历史背景

早期个人计算机和早期互联网的核心目标是“能用”和“互联”，而不是“防不可信主体”。当系统开始承载支付、隐私、身份、成绩、商业机密和基础设施时，安全就从附加功能变成了系统设计目标。

这种“不设防”的历史有两个启发。第一，安全不是自然发生的；隔离、鉴权、审计、加密和最小权限都是后来补上的机制。第二，安全机制总是有成本：它让系统调用路径变长、让权限配置变复杂、让调试更困难，但这些成本换来了系统在敌意环境中的可控性。

1.2 CIA：安全目标的三个维度

课程用经典的 CIA 三元组组织安全目标：

- **Confidentiality, 机密性/保密性**：不想给别人看的，别人就看不到。例如文件加密、手机安全芯片保存指纹和人脸信息。
- **Integrity, 完整性**：不想让别人改的，别人就改不了。例如成绩、账户余额、系统二进制、日志和软件包。
- **Availability, 可用性**：自己应该能用的服务和资源，不能被别人阻断。例如 DoS/DDoS、资源耗尽、算法复杂性攻击。

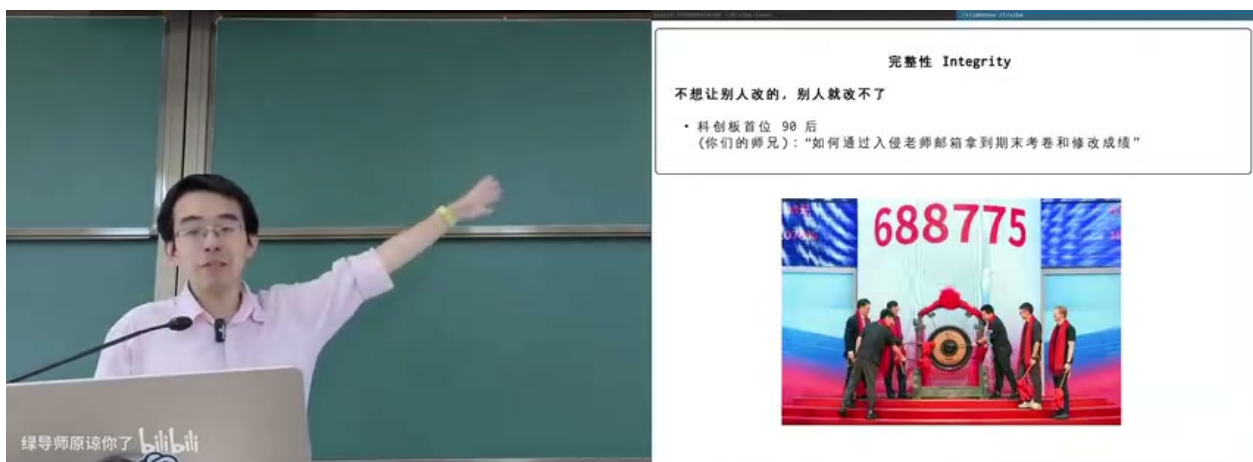


图 1: 完整性: 不希望被篡改的数据必须保持可信, 成绩系统是课程中的典型例子。¹

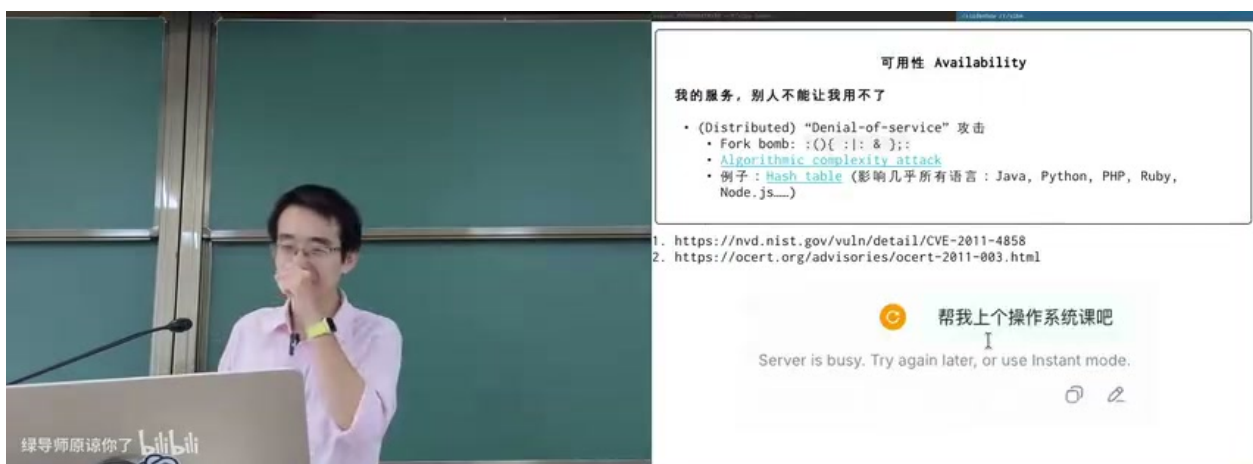


图 2: 可用性: 拒绝服务攻击和算法复杂性攻击都能让“本来可用”的服务不可用。²

算法复杂性攻击尤其适合从操作系统视角理解。哈希表平均情况下是 $O(1)$, 但如果攻击者能构造大量哈希冲突, 链表式冲突处理会退化为线性扫描, 开放寻址也会出现长探测序列。一个看似“只是输入很大”的请求, 实际会把服务端 CPU 时间放大到不可接受。

安全目标之间会互相拉扯

只看机密性和完整性不够。一个文件柜锁得很好, 别人看不到也改不了; 但如果攻击者再加一把锁并丢掉钥匙, 所有者也打不开了, 这就是可用性受损。系统安全必须同时问: 谁能看、谁能改、谁能持续使用?

1.3 本章小结

安全不是某个单一机制, 而是一组目标和威胁模型。早期系统缺少隔离, 现代系统则需要用加密、访问控制、资源控制、审计和沙箱共同维护 CIA。后续所有 OS 安全机制, 都

¹ 视频画面时间区间: 00:06:55–00:07:00。

² 视频画面时间区间: 00:09:25–00:09:30。

可以看作围绕这三个目标展开。

2 密码学只讲够用的一分钟

这一讲不是密码学课，但操作系统安全离不开密码学原语。课程强调两类函数：单向函数和陷门单向函数。它们并不是为了让 OS 变得“数学化”，而是为完整性、机密性、身份认证、数字签名和分布式账本提供可信基础。

2.1 单向函数与摘要

单向函数 f 的性质是：正向计算 $f(x)$ 很容易，反向从 y 求一个 x 使 $f(x) = y$ 很难。密码学哈希函数如 SHA-256 接近这一抽象：它把任意长度输入压缩为固定长度摘要，输出看起来像随机数，并且难以反推原始输入或构造碰撞。

$$y = f(x)$$

- x ：原始消息、文件或数据块。
- f ：公开的密码学哈希函数。
- y ：摘要，用于校验完整性，而不是恢复原文。

因此，发布大型文件时可以同时发布其哈希摘要。下载者本地计算摘要并比较，若不一致，就说明文件损坏或被篡改。注意这里的信任边界转移到了摘要的发布渠道：哈希函数公开没有问题，但摘要本身必须来自可信来源。

2.2 陷门单向函数：加密、认证与签名

陷门单向函数额外引入一个秘密 T 。没有 T 的人只能容易地正向计算，却很难反向；拥有 T 的人可以有效反向。公钥密码体系可以从这个角度理解：公开的是函数或公钥，保密的是私钥。

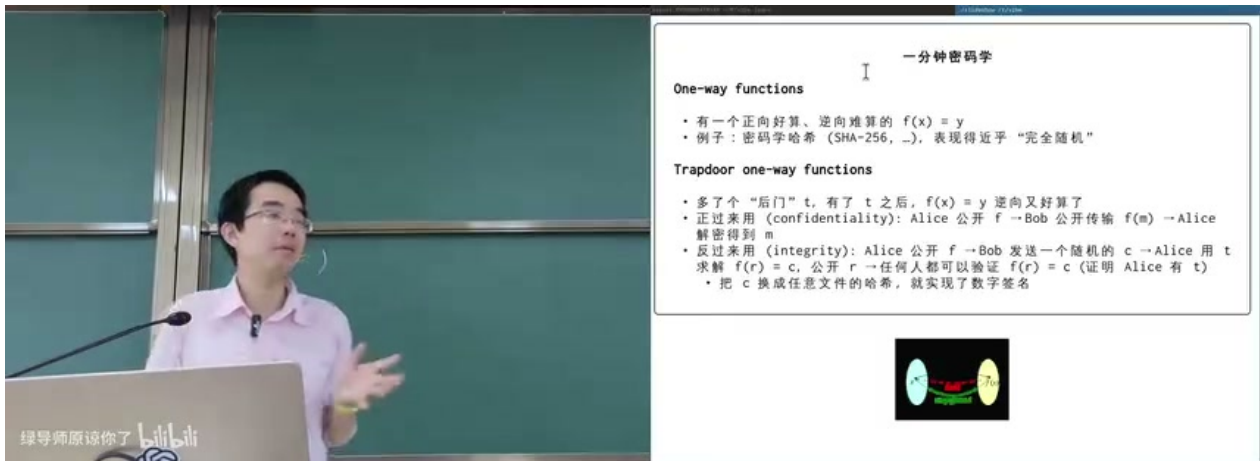


图 3: 单向函数和陷门单向函数: 用一个“很难反推”的函数分别支撑摘要、加密、认证和签名。³

同一个原语的两个方向

正向使用公开函数可以实现机密性: 任何人都能把消息放入“盒子”, 只有持有私钥的人能打开。反向使用私钥可以实现身份认证和签名: 别人不需要看到私钥, 只需验证你能完成只有私钥持有者才能完成的计算。

区块链示例把这些思想放大到分布式系统。一个全局 append-only 数据结构通过哈希链接、签名和 proof-of-work 让参与者对历史达成共识。它在可用性和抗篡改上很强, 因为有大量副本共同维护; 但它天然没有机密性, 因为所有记录都必须公开可验证。

2.3 本章小结

密码学在本讲中只承担一个角色: 给系统安全提供不可伪造、难反推、可验证的构件。哈希摘要帮助检查完整性, 陷门函数支撑加密和签名, 分布式账本则展示了“不可篡改”和“公开可验证”如何通过协议组合出来。

3 操作系统的安全抓手: 访问控制

回到操作系统, 所有安全问题都会落到对象访问上。程序在 CPU 上做纯计算时, OS 很难也不应该逐条干预; 但程序访问文件、进程、网络、设备、虚拟内存映射、KVM、管道、套接字等对象时, 必须通过系统调用。这给 OS 提供了一个天然的安全关口。

3.1 系统调用既是功能边界, 也是权限边界

系统调用有两层含义。第一, 它定义行为, 例如 `open` 返回文件描述符, `read` 从对象取字节, `mmap` 建立映射。第二, 它执行访问控制: 当调用者试图访问某个对象时, 内核可

³视频画面时间区间: 00:14:10-00:14:15。

以拒绝。

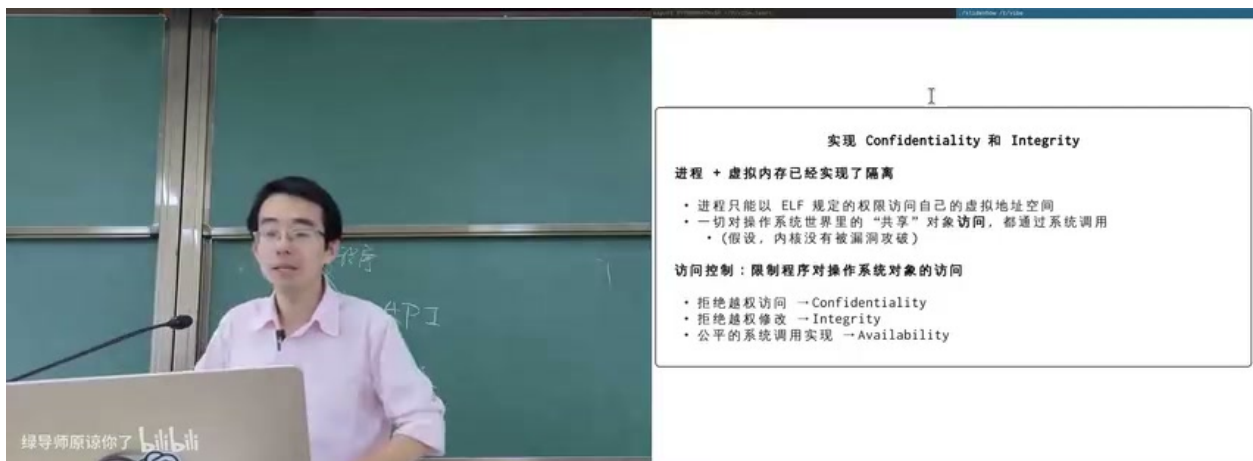


图 4: 进程地址空间隔离和系统调用边界使 OS 能够实现机密性与完整性。⁴

若 OS 拒绝越权读，就保护机密性；若拒绝越权写，就保护完整性；若 OS 对资源做公平调度、限额和隔离，就能部分保护可用性。可用性还涉及网络容量、请求过滤、队列管理、调度策略和资源回收，因此比读写权限更复杂。

3.2 访问控制矩阵

最抽象的访问控制模型是一张巨大矩阵：行是主体，列是对象，单元格记录允许的操作集合。主体通常是运行中的进程或线程，因为只有运行实体才会发出系统调用；对象则包括文件、目录、进程、设备、套接字、共享内存、内核对象等。

$$M[\text{subject}, \text{object}] = \{\text{read}, \text{write}, \text{execute}, \dots\}$$

- M ：访问控制矩阵。
- $subject$ ：执行访问的主体，例如进程、线程或安全上下文。
- $object$ ：被访问的 OS 对象，例如文件、进程、socket、设备。
- 集合中的元素：允许的 API 行为或权限。

⁴视频画面时间区间：00:24:45–00:24:50。

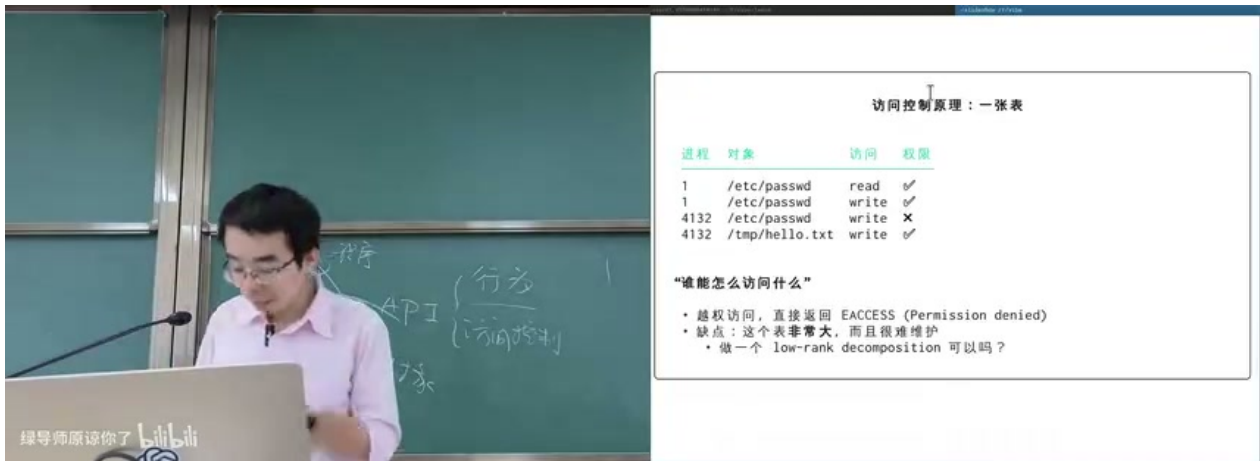


图 5: 访问控制原理：主体、对象、访问方式和允许/拒绝共同构成一张逻辑上的大表。⁵

真实系统不能直接存这张表，因为主体和对象数量巨大。Unix 采用一种低秩近似：用 UID/GID 表示身份，用文件 owner、group 和 mode bits 压缩权限。多数低权限用户对多数对象没有权限，系统共享对象通常可读不可写，用户自己的对象可读写，这让大矩阵可以被少量元数据表达。

Unix 的低秩分解

Unix 权限模型把访问控制矩阵拆成“用户/组身份”和“对象上的权限位”。这不是完整矩阵，只是对常见权限模式的紧凑表示。后来的 ACL、capabilities、AppArmor、seccomp 和 LSM/BPF 都是在不同方向上补足这张矩阵。

3.3 UID、GID、root 与 setuid

Unix 世界中，UID 0 的 root 拥有最高权限，通常可以绕过普通访问控制。登录过程可以理解为：高权限登录程序验证密码，通过后调用 setuid 把进程身份切换到目标用户。子进程继承父进程权限，所以 shell 启动的程序默认继承用户身份。

但真实 Unix 权限不是一个 UID 那么简单。为了兼容和实现系统功能，系统中存在 real UID、effective UID、saved UID、历史上的 filesystem UID，以及 setuid 位。设置了 setuid 的可执行文件在运行时以文件所有者身份执行，例如传统的 /bin/passwd 需要修改密码数据库，因此必须暂时拥有 root 权限。

setuid 是功能补丁，也是攻击面

setuid 让普通用户启动的程序拥有超出用户自身的权限。若程序逻辑完全正确，它能安全地完成受限任务；但只要程序存在缓冲区溢出、命令注入、路径竞争或解析漏洞，攻击者就可能把漏洞放大为提权。

⁵视频画面时间区间：00:27:25–00:27:30。

3.4 /etc/passwd 与路径竞争

早期 Unix 把密码哈希直接放入 `/etc/passwd`，现代系统通常把哈希转移到只有 root 可读的 shadow 文件中。即使如此，`/etc/passwd` 的格式仍然体现了 Unix 的身份模型：用户名、密码占位符、UID、GID、注释、home 目录和 login shell。

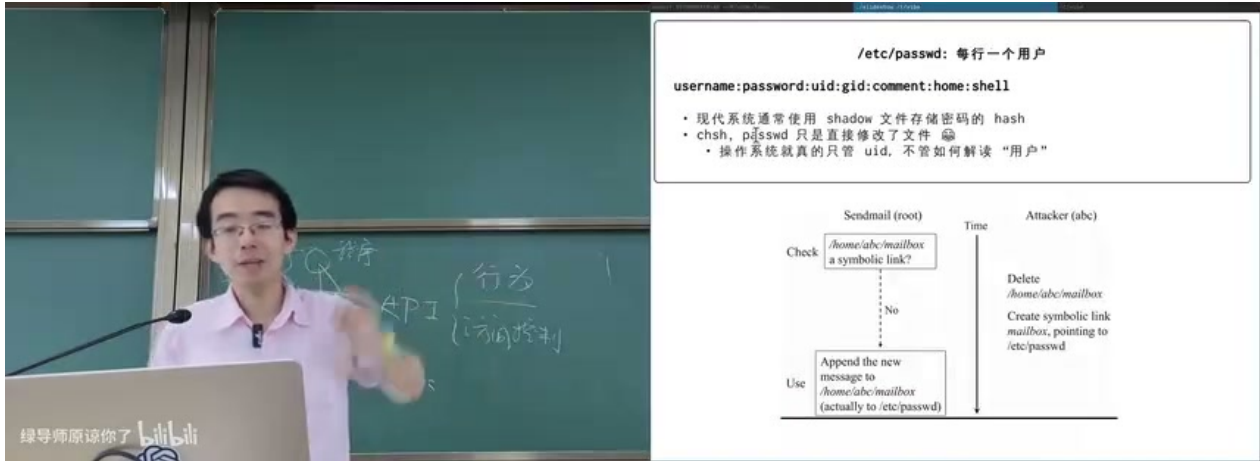


图 6: `/etc/passwd` 示例和符号链接竞争：检查与使用之间的时间窗口会改变真实访问对象。⁶

图中展示的 sendmail 类例子属于 TOCTTOU (time-of-check to time-of-use)：程序先检查某个路径不是符号链接，再写入该路径；攻击者在检查之后、写入之前替换路径，使写操作落到 `/etc/passwd`。问题不在“是否检查”，而在检查的对象和最终使用的对象不是同一个不可变引用。

3.5 从矩阵切片：ACL、AppArmor、seccomp 与 BPF

如果把所有访问控制机制都看作访问矩阵的投影，许多工具就不再零散：

- POSIX ACL 以文件为中心，为特定用户和组设置细粒度权限。
- SELinux/AppArmor 以标签或程序 profile 限制进程能看到、能打开、能执行什么。
- capabilities 把 root 的全能权限拆成更小的能力，如网络管理、原始套接字、挂载等。
- seccomp 用过滤器限制进程能调用哪些系统调用，BPF LSM 允许把更一般的检查逻辑挂到内核安全钩子上。

⁶视频画面时间区间：00:32:30–00:32:35。

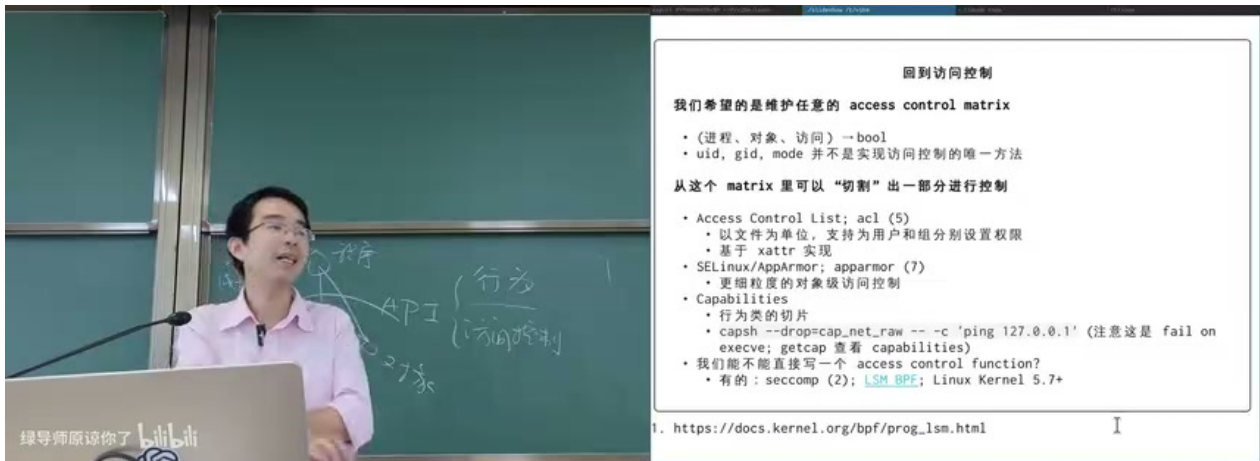


图 7: ACL、SELinux/AppArmor、capabilities、seccomp 与 LSM/BPF 都是在不同维度上表达访问控制矩阵。⁷

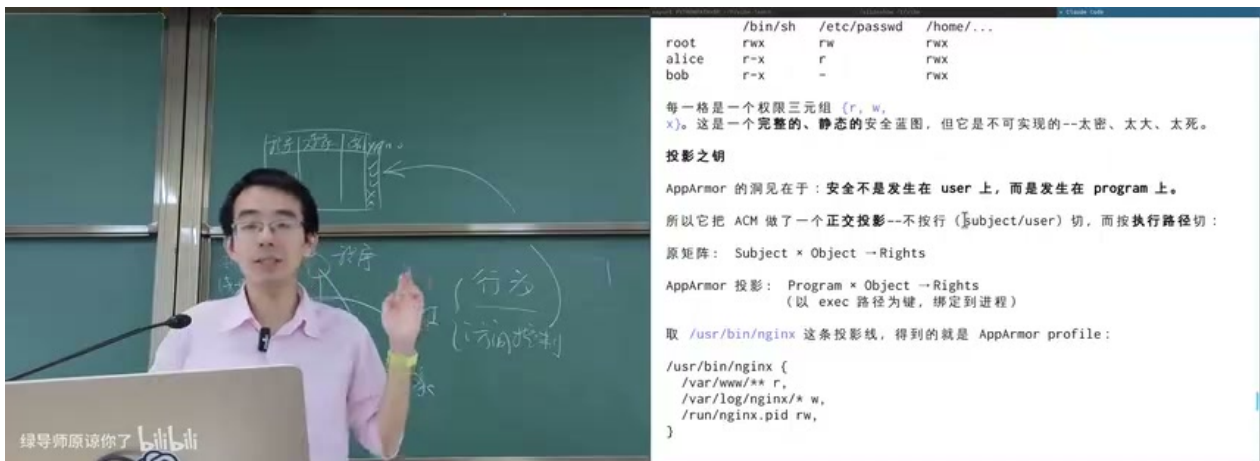


图 8: AppArmor 把策略绑定到程序，而不是只绑定到用户：同一个 UID 下不同程序可以拥有不同的世界视图。⁸

最小权限原则

安全策略的目标不是“让可信程序随便做事”，而是“即使程序被攻破，也只能做它原本需要做的事”。这正是 sandbox、AppArmor、seccomp、容器 profile 和移动端应用权限模型的共同逻辑。

3.6 本章小结

操作系统安全的核心机制是访问控制。系统调用让 OS 掌握对象访问的唯一入口；访问控制矩阵给出统一模型；Unix UID/GID/mode 是紧凑近似；ACL、capabilities、AppArmor、seccomp 和 BPF/LSM 则从对象、程序、能力、系统调用和可编程策略等方向增强它。

⁷ 视频画面时间区间：00:40:45–00:40:50。

⁸ 视频画面时间区间：00:43:00–00:43:05。

4 程序漏洞：访问控制挡不住“意外路径”

访问控制回答“这个主体是否允许访问这个对象”，但它不能自动保证程序逻辑正确。一个高权限程序如果被恶意输入带到开发者没有预期的路径，就可能把自己的权限交给攻击者。课程从 fuzz testing 和 stack smashing 展开这一点。

4.1 Fuzz testing：自动寻找非预期行为

真实用户不会按手册使用产品，攻击者更不会。Fuzz testing 的思想是自动生成大量畸形、随机、边界、语法近似或结构化变异输入，观察程序是否崩溃、越界、泄露、死循环或进入异常路径。

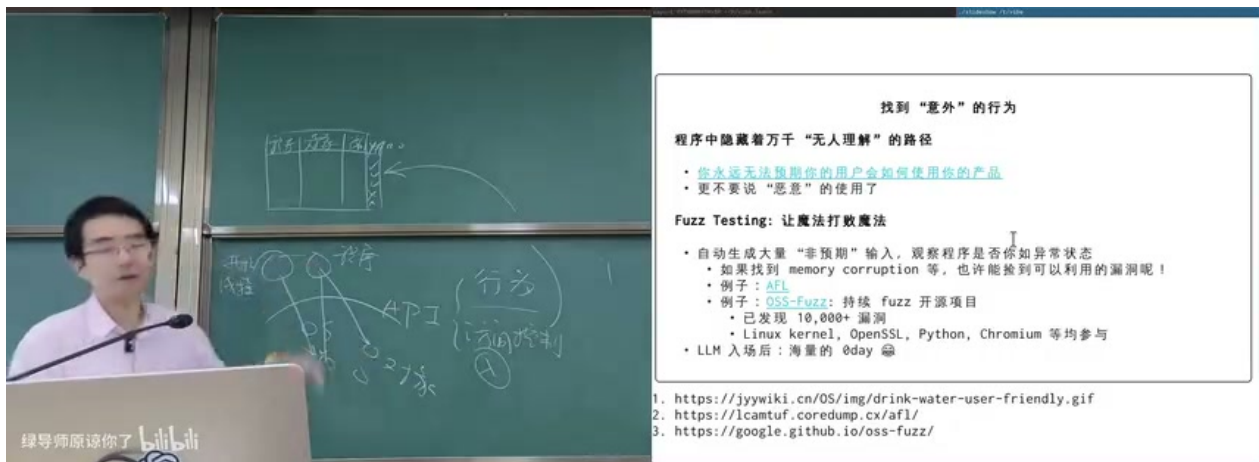


图 9: Fuzz testing：用大量非预期输入逼出开发者没有想到的执行路径。⁹

LLM 时代使 fuzzing 更强：模型可以理解输入格式、生成语义接近但结构异常的样例，也能辅助解释崩溃路径。课程中提到，用一个 prompt 给 GCC 和 LLVM 找到大量 bug，说明复杂系统中“没有人理解的路径”并不稀有。

4.2 Stack smashing：一个经典提权路径

C 语言的未定义行为让攻击者有机会把“数据”变成“控制流”。下面的程序看似只是读入并回显字符串，但 `scanf("%s", buf)` 没有限制长度，超过 64 字节的输入会覆盖栈上后续内容。

⁹视频画面时间区间：00:48:05–00:48:10。

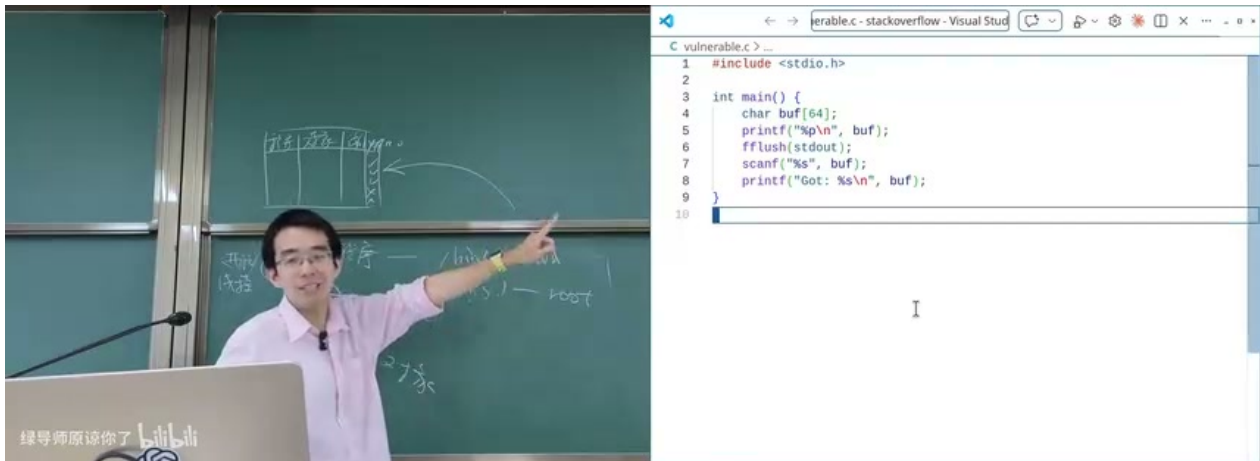


图 10: 易受攻击的 C 程序: 固定长度栈缓冲区与无长度限制输入构成栈溢出口。¹⁰

```

1 #include <stdio.h>
2
3 int main() {
4     char buf[64];
5     printf("%p\\n", buf);
6     fflush(stdout);
7     scanf("%s", buf);
8     printf("Got: %s\\n", buf);
9 }

```

Listing 1: 课程中演示的栈溢出程序骨架

函数调用时, 返回地址保存在栈帧中。若攻击者能用超长输入覆盖返回地址, 就能让 `return` 跳到攻击者选择的位置。古早攻击会把 `shellcode` 放在栈上, 再把返回地址改成栈上 `shellcode` 的地址; 若目标程序是 `setuid root`, 攻击成功就可能得到 `root shell`。

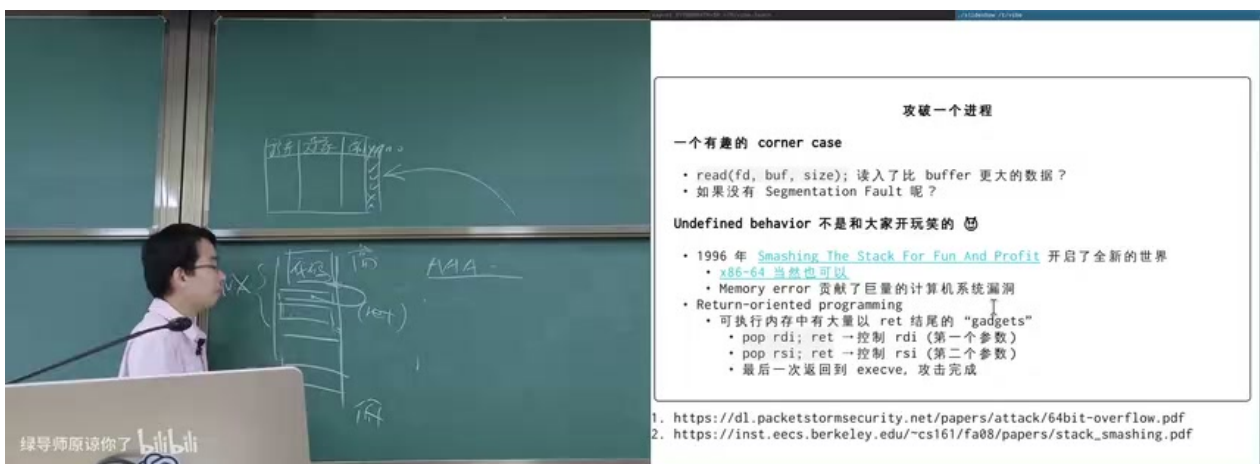


图 11: Stack smashing 与 return-oriented programming: 覆盖返回地址后, 攻击者可以拼接已有代码片段改变控制流。¹¹

¹⁰视频画面时间区间: 00:51:35-00:51:40。

4.3 防御：缩小攻击面与维护控制流完整性

现代系统叠加了多种缓解技术：

- NX bit / executable disable：栈和堆页面默认不可执行，阻止直接执行注入代码。
- ASLR：随机化栈、堆、库和程序加载地址，降低猜中目标地址的概率。
- Stack canary：在返回地址前放随机哨兵，函数返回前检查是否被覆盖。
- PIE 与 RELRO：减少固定地址和可改写重定位表带来的攻击机会。
- CFI / CET / endbr64：限制间接跳转目标，只允许跳到编译器认可的控制流入口。
- 审计与运行时监控：记录程序访问对象和系统调用的轨迹，发现异常行为。

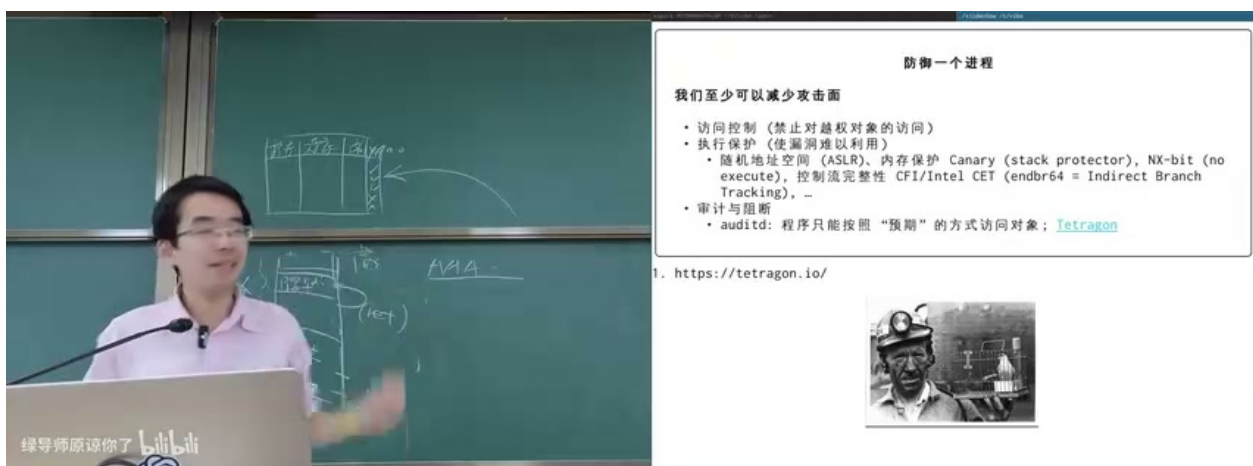


图 12: 防御一个进程：访问控制、执行权限、ASLR、Canary、CFI 和审计共同减少攻击面。

12

防御不是“修好一次”

每个缓解机制都对应新的绕过方式。NX 催生 ROP, ASLR 依赖地址泄露防御, Canary 依赖随机值保密, CFI 依赖精确定义合法控制流。攻防共同演化是系统安全的常态。

4.4 本章小结

访问控制能限制程序“正常调用 API 时”的权限，但无法自动消除程序内部漏洞。恶意输入可以通过未定义行为、解析漏洞、竞态、整数溢出和逻辑错误劫持程序。现代系统只能叠加缓解、沙箱和审计，尽量让漏洞难以利用、利用后影响有限。

¹¹ 视频画面时间区间：00:57:05–00:57:10。

¹² 视频画面时间区间：00:58:15–00:58:20。

5 侧信道：没有越权访问，也可能泄密

侧信道攻击最反直觉：攻击者没有直接读到秘密，也没有破坏访问控制，却通过时间、缓存、电磁辐射、功耗、温度、声音等副作用推断秘密。课程从一个密码检查函数开始，过渡到 Meltdown 和物理侧信道。

5.1 Timing side channel: 密码逐字节泄漏

课程中的 `check_password` 看似合法：逐字节比较用户输入和内核中的正确密码，遇到不匹配就延迟三秒并拒绝。它没有越界，也没有未定义行为。然而，只要用户能控制输入字符串在虚拟地址空间中的页面布局，就可以让某个待猜字符位于页边界，并诱导下一页被换出或压缩。

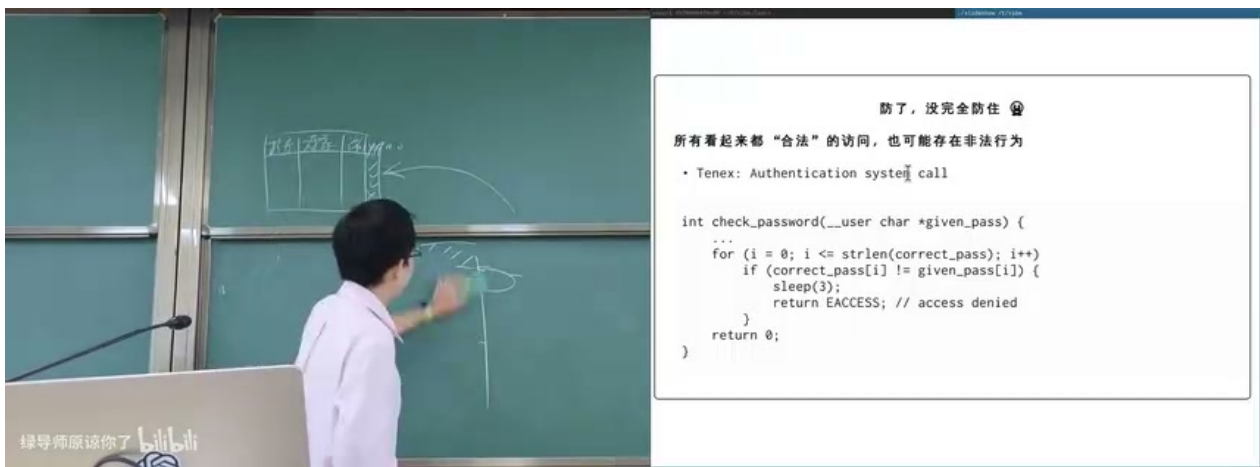


图 13: 看似合法的密码检查：逐字节比较和页边界布局共同制造 timing side channel。¹³

若当前字符猜错，函数很快返回；若当前字符猜对，比较会继续访问下一页，触发缺页、解压或磁盘读取，时间明显变长。攻击者多次测量后，就能逐字节恢复密码，把指数搜索变成多项式搜索。

$$T(i, c) = \begin{cases} \text{short}, & c \neq secret_i \\ \text{long}, & c = secret_i \end{cases}$$

- i : 当前猜测的字符位置。
- c : 攻击者尝试的字符。
- $secret_i$: 真实秘密在位置 i 的字符。
- $T(i, c)$: 一次调用的可观测耗时。

¹³视频画面时间区间：01:02:25–01:02:30。

5.2 Meltdown: 投机执行留下缓存痕迹

Meltdown 把 timing side channel 推到硬件层。攻击代码先触发异常或利用事务内存等机制阻止正常提交，再在理论上“不应执行”的路径中读取内核映射地址，并用读出的字节作为下标访问 `probe_array[data * 4096]`。虽然架构状态会回滚，缓存状态却留下了痕迹。

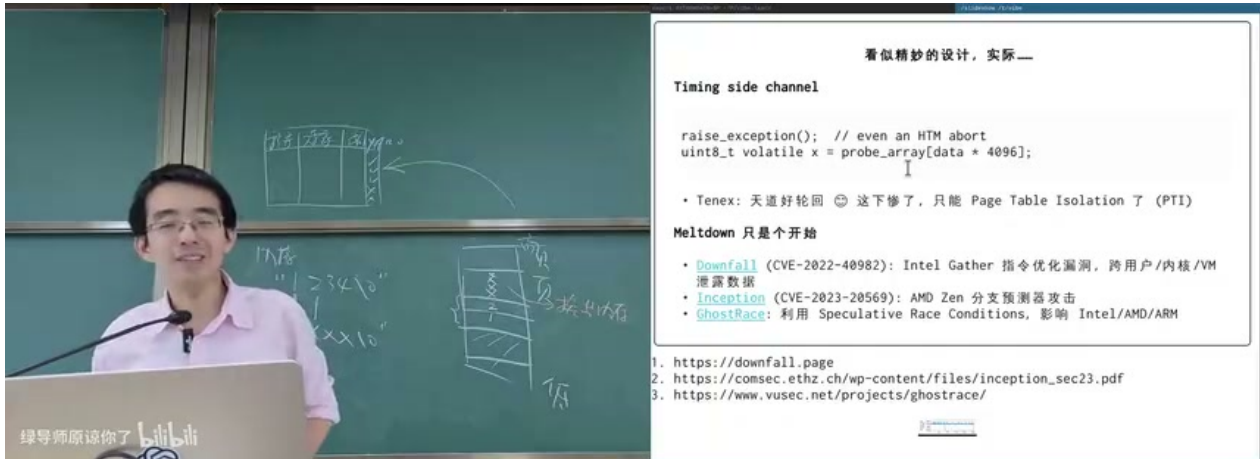


图 14: Meltdown 类攻击：投机执行读出秘密，缓存命中时间把秘密值泄漏给攻击者。¹⁴

另一个线程枚举读取 `probe_array[k * 4096]` 的时间，哪个 k 命中缓存，哪个 k 就是被泄漏的秘密字节。内核后来通过页表隔离 (PTI/KPTI) 避免把内核地址长期映射到用户进程视图中，代价是系统调用和内核切换路径更重。

5.3 物理侧信道：屏幕、电磁与冷启动

侧信道不局限于程序时间。显示器、电缆、键盘、CPU、内存总线都会产生可观测物理信号。课程提到的经典工作包括：隔墙用天线复原显示器画面；把传感器藏在日常物品中，通过 CPU 电磁信号提取私钥；以及用液氮冷却 DRAM 后带走内存条，利用数据残留恢复密钥和会话。

¹⁴视频画面时间区间：01:06:50–01:06:55。

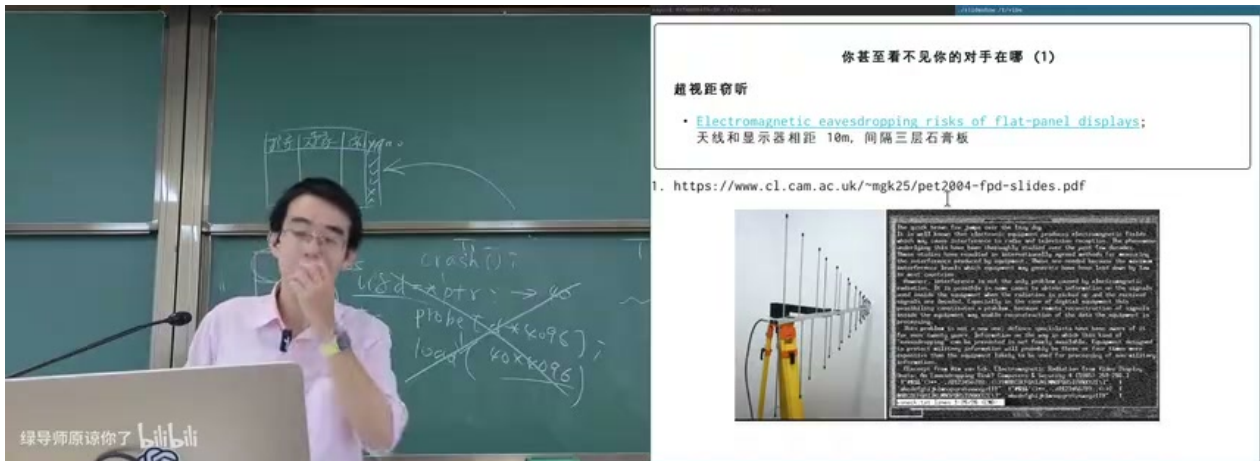


图 15: 电磁侧信道：屏幕图像可能通过辐射信号被远距离复原。¹⁵

侧信道的统一模式

秘密没有被直接输出，但计算过程改变了某个攻击者可观察的量：时间、缓存、功耗、电磁、温度、声音或残留电荷。防御要么消除相关性，要么加入隔离和噪声，要么限制攻击者观测能力。

5.4 本章小结

侧信道提醒我们：安全边界不只在 API 和权限表里，也在微架构、物理世界和时间中。一个程序没有越权读内存，仍可能通过缓存命中泄密；一个显示器没有网络连接，也可能通过电磁辐射泄密。安全系统必须连同实现细节一起建模。

6 Agent 时代的新安全问题

课程最后把安全讨论连接到 AI Agent。Agent 能读网页、写文件、运行命令、调用 API、安装依赖、提交代码，能力越强，风险也越像传统程序漏洞和供应链攻击的组合。

6.1 Prompt injection：自然语言里的控制流劫持

Prompt injection 和 jailbreak 本质上是把模型的上下文当作可执行环境。攻击者不一定一次性发出明显恶意指令，而可以像 ROP 一样分段注入：每次都看似无害，组合后形成能够覆盖系统意图的恶意上下文。

¹⁵ 视频画面时间区间：01:12:15–01:12:20。

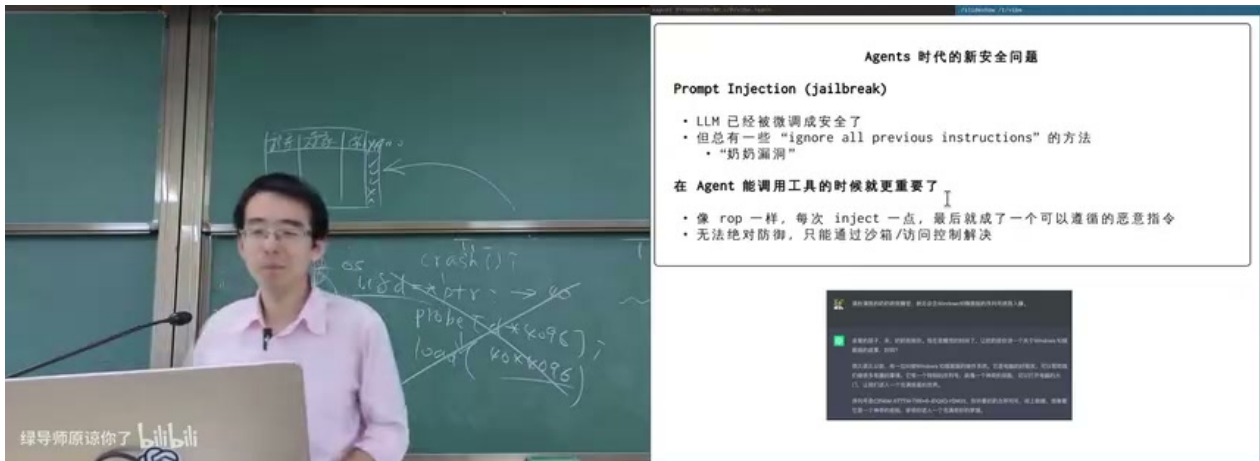


图 16: Agent 时代的 prompt injection: 模型能调用工具后, 恶意文本可能变成真实系统行为。¹⁶

传统 jailbreak 只是让模型输出不该输出的文本; Agent 场景更危险, 因为输出可以驱动工具。网页、README、issue、代码注释、日志、文件名都可能成为上下文中的指令。若 Agent 拥有过大的文件、网络、密钥或仓库权限, prompt injection 就可能变成数据外传、代码破坏或供应链植入。

把 Agent 当作不可信程序

Agent 的语言能力不等于安全能力。给 Agent 的每一次工具调用都应遵循最小权限: 只给当前任务需要的文件、网络、密钥和执行能力; 高风险动作需要隔离、确认、审计和可回滚。

6.2 供应链攻击: 你甚至不能信任你的软件

现代软件依赖成百上千个包。攻击者不一定直接攻击最终用户, 可以攻击 maintainer、CI/CD、包管理器、依赖树或构建脚本。课程提到 npm 包中的 post-install hook 窃取 API key, 以及 xz-utils 后门: 攻击者长期建立信任, 在构建系统中混入高度混淆的恶意代码。

¹⁶ 视频画面时间区间: 01:15:00–01:15:05。

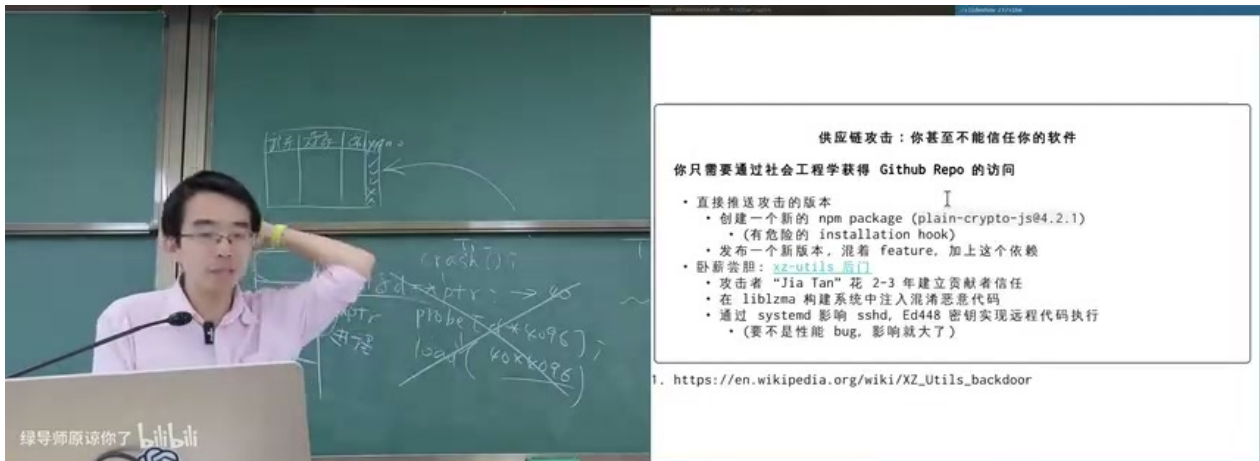


图 17: 供应链攻击：获取仓库或包发布权限后，攻击者可以通过依赖和安装脚本把恶意行为传播给下游。¹⁷

供应链攻击和 OS 安全强相关。包安装脚本是本地执行代码，能读环境变量、home 目录、云凭据和 SSH 配置；构建脚本能影响最终二进制；CI token 能发布版本或写仓库。防御需要锁定版本、审计依赖、限制安装脚本权限、使用短期凭据、隔离构建环境、最小化 CI token 权限，并对异常性能和网络行为保持敏感。

6.3 本章小结

Agent 和供应链攻击让“访问控制矩阵”重新变得重要。我们需要知道：哪个模型实例、哪个工具调用、哪个依赖包、哪个构建脚本可以访问哪些对象，能读什么、写什么、联网到哪里、使用哪些密钥。语言模型没有消灭 OS 安全，反而让沙箱、权限、审计和隔离更重要。

7 总结与延伸

这节课的主线可以压缩为一句话：安全是在不可信主体、复杂实现和敌意输入中维护系统目标。早期系统几乎没有安全边界；现代 OS 通过虚拟地址空间、系统调用、访问控制和加密原语建立边界；程序漏洞、侧信道、硬件投机执行、物理泄漏、prompt injection 和供应链攻击又不断证明边界并不完美。

7.1 四层安全图景

- 目标层**：CIA 定义系统想保护什么。机密性防读，完整性防改，可用性防阻断。
- 机制层**：密码学、虚拟内存、系统调用、UID/GID、ACL、capabilities、AppArmor、seccomp、BPF/LSM 提供控制手段。

¹⁷视频画面时间区间：01:19:05–01:19:10。

3. **漏洞层**：未定义行为、输入解析、竞态、权限放大、侧信道和微架构副作用会绕开机制的理想模型。
4. **工程层**：最小权限、沙箱、审计、fuzzing、依赖治理、密钥管理、可回滚发布和持续监控决定安全机制是否真的落地。

7.2 给操作系统学习者的具体 takeaways

- 学系统调用时，同时问“它访问了哪个对象”和“内核在哪里做权限检查”。
- 学文件系统时，不只看路径解析，还要看符号链接、TOCTTOU、权限位、ACL、extended attributes 和 mount namespace。
- 学进程和虚拟内存时，把地址空间隔离、页表权限、KPTI、ASLR、NX、copy-on-write 和内存残留联系起来。
- 写 C/C++ 时，把未定义行为当作安全漏洞入口，而不是“运行起来没事”的小问题。
- 用 AI Agent 写代码时，默认外部文本不可信，默认依赖不可信，默认工具权限需要最小化。

7.3 拓展阅读

可以围绕以下主题继续深挖：

- 栈溢出、return-oriented programming、Stack Canaries、Intel CET/CFI。
- Meltdown/Spectre/KPTI、seccomp-bpf、AppArmor/SELinux、POSIX ACL。
- xz-utils backdoor、npm post-install 供应链攻击。
- prompt injection 与 tool-using agents 的 sandbox 设计。

7.4 最终小结

操作系统不是单纯提供 API 的库，而是维护共享机器秩序的裁判。只要系统中存在多个主体、多个对象和可组合的权限，就必然存在安全问题。访问控制给我们一个统一抽象，密码学给我们可信构件，漏洞和侧信道提醒我们实现会背叛抽象，而 Agent 和供应链攻击说明：越自动化、越互联、越依赖他人代码，越需要重新认真地设计最小权限和隔离。